TODOs — vol-surface-engine

Phase 1 : SSVI, acquisition de données, calibration. Backlog de travail, à cocher sur papier.

État au 10 novembre 2026 — mesuré, pas estimé.

Table des matières

- 0. Comment utiliser ce document
- 1. Vue d'ensemble
- 1bis. Les bloquants, dans l'ordre — à faire avant le reste
- 2. Chantier A — Raccorder le moteur à la base (S2)
- 3. Chantier B — La convention de prix (S3)
- 4. Chantier C — Structure par terme et PDE (S4)
- 5. Chantier D — Qualité et dettes de code
- 6. Chantier E — Exploitation
- 7. Chantier F — Maîtrise du code (le tien)
- 8. Décisions encore ouvertes
- 9. La prochaine session
- 10. Journal
- Annexe — Définition de « fini »

0. Comment utiliser ce document

Trois règles, et il devient un outil au lieu d'une liste.

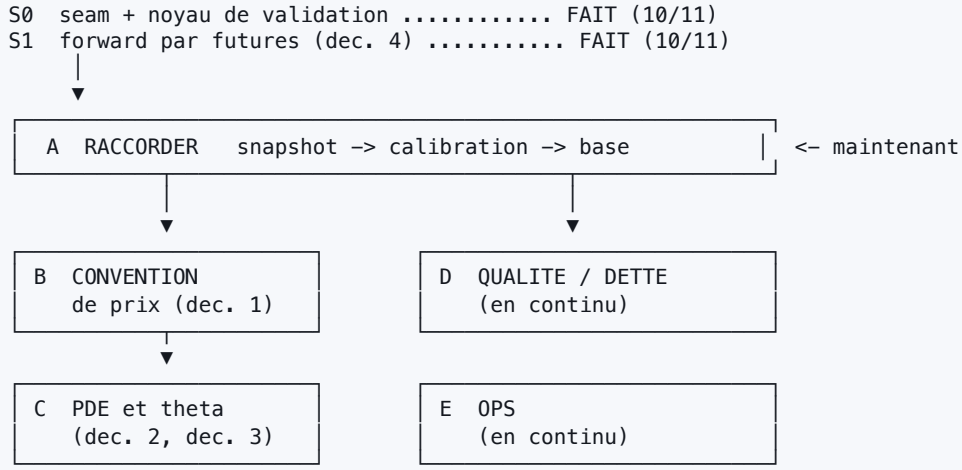
1. **Une tâche n'est finie que si son critère d'acceptation est mesuré.** « Brancher le snapshot » n'est pas fini ; « le fit BTC sort avec 11 échéances et un RMSE par échéance » l'est.
2. **Un identifiant par tâche, cité dans le commit.** `A3`, `D5`. Dans six mois, `git log --grep A3` te dit tout.
3. **On coche au crayon, et on date.** Une tâche qui recule trois semaines est un signal : soit elle est mal découpée, soit elle n'est pas vraiment prioritaire.

Les colonnes d'effort sont en sessions de travail (une session ≈ 2 h), pas en jours-homme.

Taille	Signification
S	une session, sans surprise
M	deux à trois sessions, un fichier à comprendre avant
L	une semaine ou plus, ou une décision à prendre d'abord

---

1. Vue d'ensemble



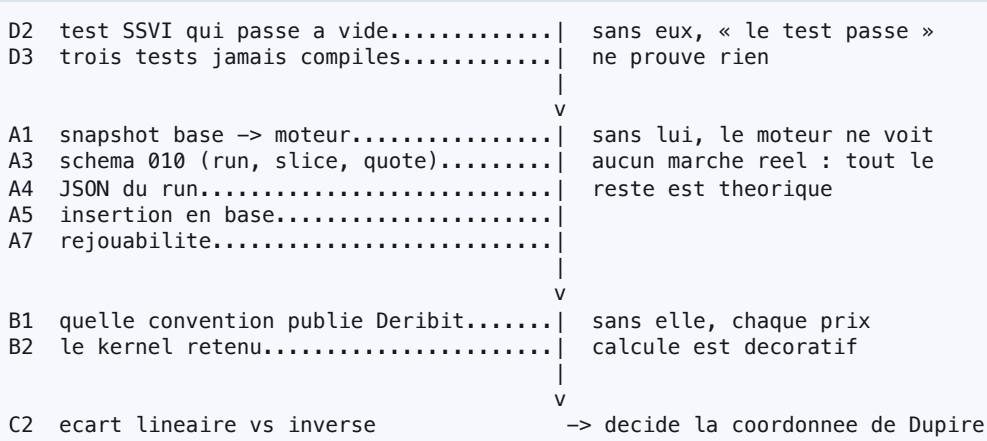
Progression

Chantier	Tâches	Faites	Reste
A — Raccorder (S2)	7	0	7
B — Convention de prix (S3)	4	0	4
C — PDE et structure par terme (S4)	4	0	4
D — Qualité et dettes de code	9	2	7
E — Exploitation	5	3	2
F — Maîtrise du code (toi)	10	0	10
Total	**39**	**5**	**34**

Les cinq premières tâches de chaque chantier sont classées : **BLOQUANT** (rien ne se mesure sans elle), **UTILE** (améliore un chiffre), **CONFORT** (ni l'un ni l'autre).

1bis. Les bloquants, dans l'ordre — à faire avant le reste

Dix tâches sont marquées BLOQUANT, mais elles ne sont pas indépendantes : l'ordre est imposé. Le détail de chacune (pourquoi ça bloque, les maths, où changer) est dans `docs/tutoriel_bloquants.pdf`.



Ordre	Tâche	Estimation	Ce qui devient possible
1	D2	S	les critères des 9 autres deviennent vérifiables
2	D3	S	le solveur d'IV est protégé d'une régression
3	A1	M	la base entre dans le moteur
4	A3	S	un run peut exister
5	A4	S	un run est un artefact portable
6	A5	S	un run est relisible en SQL
7	A7	S	un run est reproductible
8	B1	M	on sait ce qu'on price vraiment
9	B2	S	le moteur price juste
10	C2	S	la PDE devient un choix mesuré

Règle : **une session = une tâche, et la tâche se termine quand son critère d'acceptation est mesuré**, pas quand le code compile. Les tâches UTILE et CONFORT ne commencent qu'après le 10.

---

2. Chantier A — Raccorder le moteur à la base (S2)

Objectif du chantier : **un fit rejouable, produit par la base, écrit dans la base.** Aujourd'hui le moteur lit un CSV hérité et n'écrit rien. Tant que ce n'est pas fait, tout le reste est théorique.

☐ A1 — Constructeur de snapshot (C++) — BLOQUANT — M

Écrire `SurfaceSnapshot loadSnapshot(const SurfaceSnapshotSpec&)` qui lit les quotes d'un `observed_at` et construit `k_grid`, `T_grid`, `iv`.

- Forward : `MarketForwardCurve` alimenté par les points de `gold.v_forward` (S0, écrit), taux interpolé pour les échéances sans future.
- k = ln(K/F(T)), poids = 1/vega (voir A2).
- **Critère :** sur BTC au 10/11, le snapshot contient 11 échéances et ≥ 3 000 quotes, et `k_fwd` recalculé en C++ égale celui de la vue à 1e−12.
- Fichiers : `src/model/Snapshot.cpp`, `include/model/Snapshot.hpp`.

☐ A2 — Pondération du fit — UTILE — S

`SurfaceSnapshot.weight` existe mais n'est jamais rempli : le fit traite une aile illiquide comme l'ATM. Poids = 1/(vega) ou 1/(spread), au choix — mais choisi, écrit, mesuré.

- **Critère :** RMSE des ailes divisé par deux, RMSE ATM dégradé de moins de 20 %.
- Fichier : `src/calibration/SSVI.cpp` (objectif).

☐ A3 — Schéma de calibration v2 — BLOQUANT — S

Les tables ont été supprimées en 004. Migration `010_calibration.sql` : `calibration_run`, `calibration_slice`, `quote_fit`, `surface_snapshot` (voir cahier §4.8).

- **Critère :** migrations appliquées sur une base neuve (001 → 010) et sur la production, sans divergence ; `migrate` rejoue proprement.

- Fichier : `sql/010_calibration.sql`.

☐ A4 — Écriture du run depuis C++ — BLOQUANT — S

Sortie JSON du runner (`--out run.json`) : paramètres, diagnostics par échéance, résidus.

- **Critère :** un JSON valide, relu en Python, avec `engine_git_sha` et `snapshot_id`.
- Fichier : `apps/calibrate_deribit.cpp`.

☐ A5 — Insertion en base (Python) — BLOQUANT — S

`python/jobs/calibrate.py` : lit le JSON, insère `calibration_run` / `_slice` / `quote_fit`.

- **Critère :** `select * from calibration_run` montre un run BTC complet, et un rejeu du même snapshot ne crée pas de doublon (`ON CONFLICT`).
- Fichier : `python/jobs/calibrate.py`.

☐ A6 — Image Docker et ordonnancement — UTILE — M

Le binaire C++ n'est pas dans l'image : le job de calibration ne peut pas tourner. Multi-stage `engine`, puis `cron` : ingest → calibrate.

- **Critère :** `docker compose run --rm engine calibrate --snapshot latest` sort un run.
- Fichiers : `Dockerfile`, `compose.yaml`, `ops/`.

☐ A7 — Rejouabilité — BLOQUANT — S

Un fit doit pouvoir être refait à l'identique depuis `snapshot_id`.

- **Critère :** deux exécutions à 24 h d'intervalle, même `snapshot_id` → mêmes paramètres à 1e−10.

---

3. Chantier B — La convention de prix (S3)

Objectif : **savoir ce que Deribit publie vraiment**, plutôt que de le supposer. Tant que ce n'est pas mesuré, tout pricing crypto est décoratif.

☐ B1 — Mesure des trois conventions — BLOQUANT — M

Sur ≥ 5 000 quotes et ≥ 10 échéances : reconstruire `mark_price` depuis `mark_iv` avec (a) Black-76 linéaire, (b) Black inverse, (c) put linéaire sur X = 1/S, strike 1/K.

- **Critère :** tableau des écarts médians et maximaux (en points de vol) par convention, par échéance, ATM vs ailes. Une convention gagne, et on sait de combien.
- Fichier : `tools/check_convention.py`.

☐ B2 — Le kernel retenu — BLOQUANT — S

Écrire `InverseKernel` (ou corriger `LinearKernel`) selon B1, avec sa réciproque `iv()`.

- **Critère :** aller-retour prix → IV → prix à 1e−12, et les prix reproduits à un tick près.
- Fichier : `src/pricing/PricingKernel.cpp`.

☐ B3 — Lever l'ambiguïté de k en base — UTILE — S

`v_surface` expose `k_logm` (spot) et `k_fwd` (forward). Après B1/B2, une seule définition doit rester lisible ; l'autre devient une colonne d'audit, pas une alternative.

- **Critère :** un seul k alimente la calibration, et une colonne dit d'où il vient.

☐ B4 — Documenter la convention dans le cours — CONFORT — S

Section « ce que publie Deribit, et pourquoi notre modèle doit s'y aligner », avec le chiffre de B1.

---

4. Chantier C — Structure par terme et PDE (S4)

☐ C1 — θ libre par maturité (déc. 3) — UTILE — M

`θ(T) = ν·T` pinçait `η/η_max` à 0,9999 sur SPX. Option B : `θ_i` par échéance, contraint monotone (calendaire).

- **Critère :** erreur ATM par échéance divisée par deux sur BTC ; `η/η_max < 0,9`.
- Fichier : `src/calibration/SSVI.cpp`.

☐ C2 — Mesure de l'écart linéaire vs inverse pour la PDE (déc. 2) — BLOQUANT — S

Avant d'écrire une PDE dans une nouvelle coordonnée : chiffrer ce que coûte l'approximation.

- **Critère :** écart de prix (relatif) sur les ailes, par échéance ; la décision (déc. 2) est prise avec ce chiffre écrit, pas avant.

☐ C3 — PDE dans la coordonnée retenue — L — (dépend de déc. 2)

Si l'écart de C2 est sous la tolérance : documenter l'approximation, ne rien écrire. Sinon : PDE en 1/S, payoff inverse, conditions aux bords refaites.

- **Critère :** vol plate → prix analytique (inverse ou linéaire) à 1e−2.

☐ C4 — Américain : décider ou enterrer — CONFORT — S

Deribit ne cote que de l'européen. Le code américain existe (PDE + early exercise) mais ne sert personne aujourd'hui : soit on le garde documenté comme « pour la branche actions », soit on le sort du chemin chaud.

---

5. Chantier D — Qualité et dettes de code

Ces tâches ne se voient pas dans un résultat, elles se voient dans six mois.

☑ D0 — Solveur d'IV borné — FAIT (10/11)

Newton nu → bracket + Brenner-Subrahmanyam + bissection. Bornes `[intrinsèque, forward]`, `NoArbitrageViolation` distingué de `DidNotConverge`.

☑ D1 — Seam multi-conventions — FAIT (10/11)

`MarketConventions`, `ForwardCurve` (+ modèle / marché), `PricingKernel`, `SurfaceSnapshot`.

☐ D2 — `test_ssvi_synthetic` passe en affichant « TODO » — BLOQUANT — S

Un test vert qui ne teste rien est pire qu'un test absent : il occupe la ligne « 100 % passed ».

- **Critère :** le test contient un aller-retour réel et échoue si on inverse deux paramètres.
- Fichier : `tests/test_ssvi_synthetic.cpp`.

☐ D3 — Trois fichiers de test jamais compilés — BLOQUANT — S

`test_implied_vol.cpp`, `test_market_data.cpp`, `test_pde_pricer.cpp` n'ont pas de `main` : le CMake les ignore. Couverture C++ réelle = Black-Scholes seul.

- **Critère :** chaque fichier a un `main` et un cas qui échoue si la fonction est cassée exprès.

☐ D4 — `MarketData::load` avale les lignes malformées — UTILE — S

`catch (...) { continue; }` : une ligne corrompue disparaît en silence.

- **Critère :** compteur de lignes rejetées, remonté dans les stats et loggé.

☐ D5 — `NelderMead` : déclaration et définition incohérentes — UTILE — S

Le header prend `Simplex&`, le `.cpp` prend `const vector<double>&`. Rien ne les relie : la fonction n'est jamais appelée, et le jour où elle le sera, l'erreur sera un lien mort.

- **Critère :** soit aligné et testé, soit supprimé. Pas d'entre-deux.

☐ D6 — `src/main.cpp` duplique le pipeline — UTILE — M

Il refait sa propre inversion d'IV et son propre forward, en parallèle de la librairie. Deux chemins = deux vérités.

- **Critère :** `main.cpp` réduit à un point d'entrée qui appelle la librairie, ou supprimé.

☐ D7 — `etaMax` calculé deux fois — UTILE — S

Une copie dans `FitDiagnostics.cpp`, une dans `SSVI.cpp`. Si l'une bouge, le diagnostic ment.

- **Critère :** une seule implémentation, utilisée par les deux appelants.

☐ D8 — `calibrate_spx_minimal` suppose r = q = 0 en dur — UTILE — S

Il ignore `rates.csv` et `divs.csv` pourtant chargés. Acceptable en démo synthétique, dangereux ailleurs.

- **Critère :** les courbes lues sont utilisées, ou le runner est renommé pour dire sa limite.

---

6. Chantier E — Exploitation

☑ E1 — TimescaleDB épinglée, port loopback, restart — FAIT (oct.)

☑ E2 — Garde-fou disque, bases de test, fraîcheur du silver — FAIT (10/11)

☑ E3 — Rétention du bronze (fichiers horodatés) — FAIT

☐ E4 — Surveillance du fit, pas seulement de l'ingestion — UTILE — M

Aujourd'hui on surveille « la base bouge ». Personne ne surveille « le fit est encore raisonnable » : un RMSE qui double passe inaperçu.

- **Critère :** alerte si RMSE ou `η/η_max` sort d'une bande, sur les 3 derniers runs.
- Fichier : `ops/mini-alerte.sh`.

☐ E5 — Restauration testée sur une base jetable — UTILE — S

Le script existe (`mini-verif-restauration.sh`, dimanche 04:00) : vérifier qu'il a déjà produit un succès réel, et sinon le réparer.

- **Critère :** une ligne de log « restauration vérifiée » de moins de 8 jours.

---

7. Chantier F — Maîtrise du code (le tien)

C'est le chantier qui ne se voit pas dans un commit. Quatre niveaux par module ; on ne passe au suivant que quand le précédent est vrai **sans le code ouvert**.

#	Module	Expliquer	Réécrire	Casser exprès	Mesurer
F1	`blackscholes` + `ImpliedVol`	☐	☐	☐	☐
F2	`SSVI` (paramétrage, η_max, objectif)	☐	☐	☐	☐
F3	`LocalVol` + `PDEPricer` + `Tridiag`	☐	☐	☐	☐
F4	`RateCurve` / `DivCurve` / `ForwardCurve`	☐	☐	☐	☐
F5	`MarketData` / `SurfaceSnapshot`	☐	☐	☐	☐
F6	Schéma SQL (4 tables, 5 vues, unités)	☐	☐	☐	☐
F7	`load_marks` (bronze → silver)	☐	☐	☐	☐
F8	`deribit_ws` (canaux, watchdog, rotation)	☐	☐	☐	☐
F9	Le seam multi-conventions (S0)	☐	☐	☐	☐
F10	La chaîne complète, de la trame au paramètre	☐	☐	☐	☐

Ce que veut dire chaque colonne :

- **Expliquer** — le dessiner au tableau, en 5 minutes, sans notes.
- **Réécrire** — la fiche fonction du cahier remplie, puis comparée au code. Les écarts sont les vraies questions.
- **Casser exprès** — introduire un bug (inverser un signe, retirer un bracket, changer une unité) et vérifier qu'un test le voit. Si aucun test ne le voit : c'est une tâche du chantier D.

- **Mesurer** — prédire un chiffre avant de lancer (complexité, erreur, temps), puis comparer.

Exercice le plus rentable, à faire une fois : **casser `ImpliedVol` exprès** (retirer les bornes d'arbitrage) et regarder ce que la calibration fait. C'est ce qui t'apprendra pourquoi le bracket existe.

---

8. Décisions encore ouvertes

#	Question	Ce qui manque pour trancher	Où
déc. 2	Coordonnée de Dupire	la mesure de C2	cahier §5
déc. 3	`θ(T)` figé ou libre	l'erreur ATM par échéance sur BTC	cahier §5 / C1
déc. 6	Capacité et rétention	la croissance mesurée du silver	cahier §5

Décidées : **déc. 4** (futures + interpolation), **déc. 5** (reprise au présent). **déc. 1** (convention de prix) reste ouverte jusqu'à B1.

---

9. La prochaine session

À remplir au début de chaque séance : trois tâches maximum, et une seule « BLOQUANT ».

```
1. ..........................................................................
2. ..........................................................................
3. ..........................................................................

Ce que je veux savoir à la fin : ..............................................
```

---

10. Journal

Date	Tâche	Ce qui a changé	Ce que j'ai appris
10/11	S0, S1, D1, E2	seam, futures, forward, garde-fou	un collecteur healthy ne dit rien de la chaîne

---

Annexe — Définition de « fini »

Une tâche est finie quand les cinq lignes sont vraies :

```
[ ] le critere d'acceptation est mesure, et le chiffre est ecrit quelque part (commit, doc, cahier)
[ ] un test echoue si on casse la fonction expres
[ ] le comportement est documente la ou quelqu'un le cherchera (pas dans un commentaire perdu)
[ ] aucune duplication nouvelle (une seule definition par formule)
[ ] je peux l'expliquer sans ouvrir le code (chantier F)
```